

Lab 2: Unsupervised Learning — PCA and Autoencoder Feature Representations

Objectives

By the end of this laboratory activity, you should be able to:

- Evaluate how unsupervised dimensionality reduction affects a supervised classification task.
- Compare the feature representations learned by Principal Component Analysis (PCA) and an autoencoder.
- Investigate how representation size affects information retention and classification generalization.
- Defend a model choice using experimental evidence and the characteristics of your chosen dataset.

Instructions

1. Select and examine a classification dataset

Choose a dataset with enough input features to investigate several reduced dimensions. Cite its source and describe its classification task, sample count, feature types, class distribution, missing values, and possible redundant or noisy features. Explain why dimensionality reduction may be useful for this dataset.

Separate the target labels from the input features. Labels must not be included in preprocessing inputs, PCA fitting, or autoencoder reconstruction targets.

2. Prepare the data and define the experiment

Split the dataset into **training, validation, and test sets** before fitting any preprocessing steps. Choose and justify the split proportions. Use stratification when appropriate; if observations share subjects, groups, or time dependencies, use a split that prevents related observations from leaking across partitions. Record the split and random seeds so that all configurations use the same observations.

Apply suitable preprocessing, such as missing-value imputation, one-hot encoding, standardization, or min-max scaling. Explain your choices. Fit all learned preprocessing parameters on the training set only, then apply the fitted transformations to the validation and test sets.

Let n be the number of input features **after preprocessing**. Select at least three distinct reduced dimensions k , where $1 \leq k < n$, that are valid for both methods and the available training sample count. Include a small, intermediate, and relatively large representation. For example, when $n = 20$, you might investigate $k = 2, 5, 10$. Use the same candidate dimensions for PCA and the autoencoder.

In this lab, reducing dimensions means constructing k new features from the n input features. It does not simply mean deleting original columns.

Use the partitions as follows:

- **Training:** fit preprocessing, PCA, autoencoders, and classifiers.
- **Validation:** monitor training, select hyperparameters and checkpoints, and choose the final dimension for each reduction method.
- **Test:** evaluate the three finalized configurations only after all choices are fixed. Do not use test results to tune or select models.

3. Build a classifier using all input features

Implement a neural-network classifier with at least one hidden layer and one output layer, excluding the input layer. Choose an output activation and loss appropriate for binary or multiclass classification.

Declare the architecture, activations, optimizer, learning rate, batch size, maximum epochs, stopping rule, and random seeds. Choose a primary classification metric and justify it using the task and class distribution. Use the same primary classification metric for every configuration. Report suitable supporting classification metrics and a confusion matrix as well.

Train and validate this classifier using all n preprocessed features. This is your **baseline**.

For the comparisons below, train a fresh classifier for every representation. Keep the hidden-layer architecture, training settings, and stopping rule fixed; change the input width to match the representation. Use at least three predefined

random seeds for each candidate configuration and report the mean and standard deviation of classification scores. Repeat stochastic training, including autoencoder training, for each seed. These repeats measure training variability on the fixed split, not uncertainty across possible dataset samples.

4. Apply PCA

Study and implement **Principal Component Analysis (PCA)**. For each candidate k :

- Fit PCA on the preprocessed training inputs only, then transform the validation inputs with the fitted PCA model.
- Train and validate a fresh classifier using the k principal-component scores as its input features.
- Record validation classification performance using your declared metric.

Inspect the component loadings or coefficients to explain which original input features contribute to selected components. Use validation classification performance to select the final k ; if candidates perform similarly, state and apply a predefined preference, such as choosing the smaller representation.

5. Train an autoencoder

Design an autoencoder that maps the same preprocessed inputs to a bottleneck of k latent features and reconstructs all n inputs. Include nonlinear activations so that you can investigate a nonlinear representation. Justify the decoder output activation and reconstruction loss using the input scale and feature types.

For each candidate k :

- Train the autoencoder on training inputs, using those inputs as reconstruction targets. Monitor validation reconstruction loss to select its checkpoint.
- Document its architecture, activations, optimizer, learning rate, batch size, epochs, stopping rule, and any regularization. Keep the design and training procedure consistent across dimensions apart from changes required by the bottleneck width.
- Freeze the trained encoder and extract latent features for the training and validation sets. Train a fresh classifier on these features without updating the encoder through classification loss.
- Record validation classification performance using your declared metric.

Select the final bottleneck dimension using validation classification performance and the same tie-breaking rule used for PCA. Do not select a representation solely because it has the lowest reconstruction loss.

Use reconstruction loss internally to train the autoencoder and select its checkpoint. Evaluate the usefulness of its latent features through the downstream classifier's classification metric.

6. Compare and report your findings

Plot validation classification performance against k for both methods. Include the baseline classification score as a reference and show variability across seeds. Label the metric and indicate whether higher or lower values are better.

After freezing all choices, evaluate the baseline and the selected PCA and autoencoder configurations on the untouched test set for each predefined seed. Report all runs rather than selecting the best seed. Include a confusion matrix for each configuration using a seed designated before test evaluation, or clearly explain another aggregation method.

Complete the final comparison table below using the same primary classification metric for all three configurations. Replace “primary classification metric” in the headers with its name, such as accuracy, macro F1, or balanced accuracy. Report classification scores as **mean $\pm$ standard deviation** across seeds.

Configuration	Input dimensions	Validation primary classification metric	Test primary classification metric
All-feature baseline	n		
PCA + classifier	Selected k		
Autoencoder + classifier	Selected k		

Submit a runnable notebook or scripts, dataset access instructions, package versions, and a report containing your experimental settings, candidate-dimension results, required plots, final comparison, and answers to all five guide

questions. Support conclusions with specific results. A well-supported finding that dimensionality reduction does not improve classification is valid.

Guide Questions

Answer each question using your chosen dataset and numerical or visual evidence from your experiments. Distinguish observations from proposed explanations, and acknowledge what your experiment cannot establish.

1. **Which configuration performed better, and why for this dataset?** Compare the baseline, PCA, and autoencoder using your primary metric, variability across seeds, and class-specific errors. Relate the outcome to at least two observed dataset properties, such as feature correlation, sample size, class imbalance, noise, or nonlinear structure. Does the evidence support a clear winner, a practical tie, or a trade-off? Explain why the ranking should not automatically generalize to other datasets.
2. **Are fewer dimensions better for PCA and for an autoencoder?** Use your classification-performance-versus-dimension curves to identify where compression helps, hurts, or makes little difference for each method. Explain the tension between removing redundancy or noise and discarding useful class information. How does restricting a linear subspace differ from restricting a learned nonlinear bottleneck? Defend a final k for each method and explain why the smallest representation or the best reconstruction need not yield the best classifier.
3. **What do the reduced features actually represent?** Distinguish feature extraction from selecting original columns. Use PCA loadings and at least one concrete autoencoder probe, such as latent-feature correlations with inputs, class-colored latent plots, or decoded latent perturbations, to investigate what each representation preserves. Explain how PCA's variance ordering and orthogonality differ from the autoencoder's learned latent coordinates. What can and cannot be inferred about individual latent features, and why is one latent coordinate not necessarily equivalent to one original attribute?
4. **Does preserving the input preserve the information needed for classification?** Use your classification results across dimensions and class-specific errors to discuss whether compression preserved useful class information. Construct a plausible example grounded in your dataset where a low-variance feature is highly predictive or a high-variance feature is irrelevant. Explain conceptually why retaining high variance in PCA or reconstructing inputs well with an autoencoder does not guarantee strong classification performance. Explain why neither unsupervised objective directly optimizes class separation, and how scaling or mixed feature types could alter which information the method preserves.
5. **What evidence would challenge your explanation of the winner?** Propose and run one controlled follow-up experiment using training and validation data only, such as changing training sample size, removing a documented redundant feature group, or adjusting autoencoder regularization. State a hypothesis and predicted effect on both methods before running it. Keep the other relevant factors fixed, report the change in the validation classification metric, and explain whether it supports your account of underfitting, overfitting, representation capacity, or dataset structure. Identify a remaining confounder and describe what further evidence would resolve it. Do not retune against the test set.

Rubric — 95 Points

Criterion	Full-credit requirements	Maximum points
Dataset understanding and preparation	Dataset source and characteristics are documented and the choice is justified (5); preprocessing is appropriate and fitted on training data only (5); splits, seeds, and test isolation are correct and explained (5).	15
Baseline and comparison design	Classifier, hyperparameters, and metrics are appropriate and documented (4); matched dimensions, fixed classifier settings, and at least three seeds support a fair comparison (6).	10
PCA implementation and analysis	PCA is fitted and applied correctly across candidate dimensions (4); loadings, classification performance across dimensions, and dimension selection are correctly analyzed (6).	10
Autoencoder implementation and analysis	Architecture, training, checkpoint selection, and frozen-encoder feature extraction are correct (5); candidate dimensions, classification performance, and dimension selection are documented and analyzed (5).	10
Results and reproducibility	Candidate results, required plots, confusion matrices, final classification-metric table, and variability are complete and labeled (6); submitted code, data access, versions, and settings allow the work to be reproduced (4).	10
Guide questions	Five answers, each worth 8 points, demonstrate evidence-based reasoning, conceptual accuracy, and dataset-specific interpretation using the scoring breakdown below.	40
Total		95

For each procedural subcriterion in the first five rows, award 100% of its points when complete and correct, 75% for minor omissions, 50% for substantial gaps with some valid work, 25% for a minimal relevant attempt, and 0% for absent or invalid work. Retain fractional points until summing the final score.

Score **each guide question out of 8** as follows:

- **Evidence (0–3):** 3 for specific, relevant experimental evidence that supports the reasoning; 2 for relevant but incomplete evidence; 1 for vague references to results; 0 for no supporting evidence. For Question 5, full credit requires the executed controlled experiment.
- **Conceptual reasoning (0–3):** 3 for an accurate explanation of the relevant PCA and autoencoder mechanisms and trade-offs; 2 for mostly correct but incomplete reasoning; 1 for superficial reasoning or major misconceptions; 0 for absent or incorrect reasoning.
- **Dataset connection and critical judgment (0–2):** 2 for explicitly connecting the explanation to the chosen dataset and acknowledging limitations or alternative explanations; 1 for addressing only one of those elements; 0 for addressing neither.

Grades reflect the quality of the experiment and reasoning, not whether PCA or the autoencoder achieves the highest score. Report the final grade as **earned points / 95**.